
Predictions of Smartphone Battery Behaviors and Recommendations for Cellphone Users

Smartphones have become indispensable tools in modern life, yet their battery behavior remains a major concern for many smartphone users. Power consumption is a complex dynamic process influenced by hardware usage, software behavior, environmental conditions, and the battery's own electrochemical state. In this paper, we develop a behavior-aware and physics-informed modeling framework to model lithium-ion battery discharge, predict battery discharge dynamics and time-to-empty (TTE), and provide actionable recommendations to extend battery runtime and longevity.

We first developed a multivariate linear regression model decomposing battery drain into key user-controllable factors: screen usage, brightness, processor load, network activity, and background pressure. Using Ridge Regression to handle multicollinearity, our model achieved a high explanatory power ($R^2 \approx 0.94$), with feature ablation and sensitivity analyses identifying screen on time and processor load as the dominant contributors of power drain.

To capture the non-linear voltage collapse near battery depletion (the "avalanche effect"), we augmented the linear model with a Weibull-based impedance correction. In parallel, we integrated a Second-Order Thevenin Equivalent Circuit Model to simulate transient voltage responses and diffusion effects. Moreover, we incorporated a State of Health (SOH) coupling mechanism to account for capacity fade over time.

Our model was validated against real-world datasets encompassing various device brands, user behavior and other conditions. The model demonstrated strong predictive capability with a Mean Absolute Percentage Error (MAPE) of less than 10% for medium to high usage scenarios.

Perturbation testing revealed that the model is most sensitive to processor load and screen on time, confirming these as critical control variables. We successfully derived a linear prediction function for daily battery drain.

Based on our quantitative analysis, we provide the following prioritized recommendations to maximize battery runtime and longevity: reduce brightness (or initiate auto-brightness and dark mode) and screen timeout duration since screen display is the primary power consumer; Close high-intensity background apps and disable unnecessary push notifications; Prioritize Wi-Fi over cellular data (5G/4G) whenever possible, as cellular radios consume up to twice the power of Wi-Fi; Maintain the device between 15 and 28° C; Avoid deep discharging (0%) or keeping the battery at 100% for extended periods since the optimal longevity range is 40%-80% charged.

In conclusion, our study bridges the gap between statistical user behavior analysis and electrochemical battery dynamics, offering a robust tool for predicting TTE and a practical guide for users to optimize their smartphone experience.

Keywords: lithium-ion battery; State of Charge (SOC); power consumption modeling; battery ageing; Time-to-Empty prediction; user behavior analysis

Contents

1	Introduction	1
1.1	Background Information and Restatement of the Problem	1
1.2	Related Work	2
2	Model Formulation and Evaluation	3
2.1	Basic Rationale	3
2.2	Data Processing	5
2.3	Model Refinement: Physics-Informed Dynamics & Ageing	8
2.3.1	The Weibull-Augmented Impedance Model	8
2.3.2	Second-Order Thevenin Equivalent Circuit Model	9
2.3.3	The Unified Governing Equation	10
2.3.4	Ageing Integration	10
2.4	TTE Prediction	11
2.4.1	TTE Prediction	11
2.4.2	Battery Life Prediction	13
3	Sensitivity Analysis and Robustness Checks	15
4	Recommendations for Users	19
5	Further Discussion and Future Works	21
A	Tools and Software	22
B	The Codes	22
B.1	Excerpt of Linear Model Construction	22
B.2	Excerpt of Feature Ablation Experiment	23
B.3	Excerpt of Sensitivity Analysis	23
B.4	Excerpt of Battery Drain Prediction	24
C	AI Use Report	24

1 Introduction

1.1 Background Information and Restatement of the Problem

Now that smartphones have become so popular in the era of information explosion, we all have the experience where our smartphones run out of battery, which can be quite bothering.

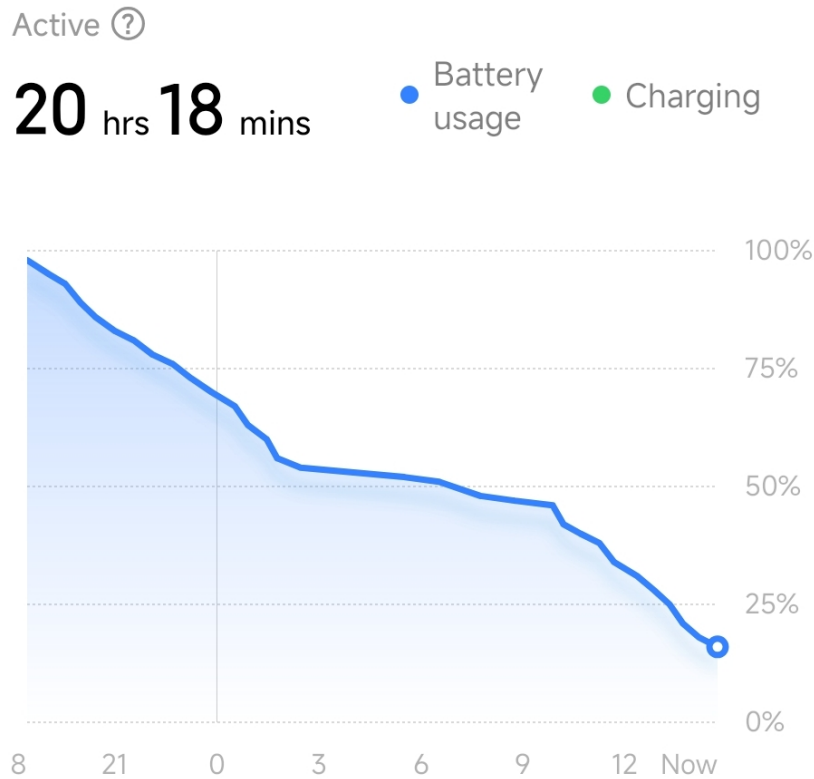


Figure 1: A real-condition SOC-t graph

This is the SOC-t graph of a Redmi K70 Pro provided by the Settings app. From 6 p.m. to 2 p.m.(the next day), SOC drops from 100% to about 20%. It is approximately a piecewise linear function. The slope goes through a steep-smooth-steep process, as the user goes to sleep in the smooth part and there is no active usage involved.

So what is the law behind the power consumption of the smartphone? Is there anything we can do to extend the time the battery lasts? In this paper, we build models to decide which are the dominant factors of power consumption and to predict time-to-empty accordingly.

Before we start, we must understand some technical terms and the basic principles how a lithium-ion battery works.

SOC (state of charge): a measure of how much energy remains in a battery compared to its full capacity, expressed as a percentage, ranging from 0% to 100%. This is the number we can directly see from the phone screen. In this paper, we assume that the SOC shown on the phone screen is precise.

time-to-empty: the estimated amount of time remaining before a battery is completely discharged.

SOH (state of health): is used to describe the health status of a battery, and is often defined by

$$SOH = \frac{E_{present}}{E_{original}}$$

where $E_{present}/E_{original}$ is the present/original capacity. Note that both energy and capacitor can be used to describe SOH and other similar quantities.

cutoff voltage V_{cutoff} represents the lower safety threshold enforced by the Battery Management System (BMS). Typically set around 3.3V for lithium-polymer batteries, this threshold triggers an automatic shutdown to prevent irreversible chemical degradation of the active materials, serving as the practical termination point for the Time-to-Empty (TTE) calculation.

OCV (open circuit voltage): represents the difference of electrical potential between the two terminals of a device (here specifically, a battery) when there is no external load connected and no electric current flows between the terminals. OCV is the most accurate parameter for estimating the State of Charge (SOC). There is a direct relationship (OCV-SOC curve) where a specific voltage corresponds to a specific percentage of remaining battery capacity.

The well known Ohm's Law for Closed Circuits tells us that when discharging:

$$V_{terminal} = OCV - I \cdot R_{internal} \quad (1)$$

working principles of lithium-ion battery:[12][13]

- Components: Cathode (e.g., LiCoO₂, NMC, LFP), anode (graphite, silicon composites), electrolyte (Li salt & solvent), separator, current collectors, cell enclosure.
- Basic electrochemical process: Charge/discharge are reversible lithium intercalation/deintercalation reactions: during discharge Li moves from anode to cathode through electrolyte; charge is the reverse. Cell potential is set by the Gibbs free energy difference of the electrode reactions: $E_{oc} \approx -\Delta G/(nF)$. Open-circuit voltage (E_{oc}) depends on electrode chemistry and SOC.
- Transport & kinetics: Electrons flow through the external circuit; Li⁺ migrate in the electrolyte and intercalate into solids. Charge transfer at the electrode/electrolyte interface introduces reaction kinetics limitations. Solid-phase Li diffusion (characterized by D_s) limits high-rate performance and causes concentration polarization inside active particles.
- Surface/interface phenomena: SEI (solid-electrolyte interphase) forms on the anode: it affects initial coulombic efficiency and internal resistance; grows with aging. Side reactions (electrolyte decomposition, metal dissolution) cause capacity fade and self-discharge.

1.2 Related Work

Existing research on battery State of Charge (SOC) estimation generally falls into two categories: traditional modeling and data-driven approaches. Foundational methods primarily rely on "Coulomb counting" [4] and software-based energy profiling [6][7]. While these techniques provide a baseline for power consumption, they often struggle to accurately model

the non-linear relationship between SOC and time under the dynamic workloads of modern smartphones [4][7].

To improve accuracy, recent studies have shifted towards machine learning methodologies. Deep learning models, such as LSTM networks [1][3] and attention-based frameworks [5], have proven effective in capturing erratic user behaviors and non-linear discharge patterns. Furthermore, research highlights that accurate runtime prediction must also account for battery degradation, as ageing and environmental factors significantly alter the discharge curve over a device's lifespan [2].

2 Model Formulation and Evaluation

2.1 Basic Rationale

If E_{cap} is the energy capacity of the battery, and $P(t)$ is the discharge power at time t , we can get the following relationships:

$$\frac{dSOC(t)}{dt} = -\frac{P(t)}{E_{cap}} \quad (2)$$

$$\int_{t_0}^t \frac{P(t)}{E_{cap}} dt = SOC(t_0) \quad (3)$$

where $SOC(t_0)$ is the initial state of charge of the battery.

Or if we use capacitor to describe:

$$SOC(t) = SOC(t_0) - \frac{1}{C_{cap}} \int_{t_0}^t I(\tau) d\tau \quad (4)$$

If we can find t^* that satisfies $SOC(t^*) = 0$, we can get the value of TTE since

$$TTE = t^* - t_0 \quad (5)$$

Hence, based on the PowerBooster technique[4][11], we proposed a model for SOC-timeline description and TTE prediction.

By collecting working voltage $V(t)$ and converting it to OCV(t), we get the discharge current by the following equation given that $f(OCV(t)) = SOC(t)$:

$$I(t) = -C_{cap} \cdot \frac{d(SOC(t))}{dt} = -C_{cap} \cdot f'(OCV(t)) \frac{d(OCV(t))}{dt} \quad (6)$$

Here C_{cap} is the rated capacitor of the battery.

Real-time power:

$$P(t) = V(t) \cdot I(t) \quad (7)$$

Thus,

$$P(t) = -C_{cap} \cdot V(t) \cdot \frac{d(SOC(t))}{dt} = -C_{cap} \cdot f'(OCV(t)) \frac{d(OCV(t))}{dt} \quad (8)$$

For each single component, we define variables to describe the working status and power level:

Table 1: meanings of the variables in equation (13)

parameter	meanings	variable type	way to be determined
$P(t)$	real-time power	continuous value	calculated from data collected with sensors
P_0	baseline power	constant value	fitting
m	total number of state variables after division	integer	defined before calculating
α_k	global power coefficient of the k^{th} component	constant value	fitting
$z_k(t)$	boolean variable of the k^{th} component	continuous/binary	collected from system api

1. **binary state component** that is either on or off. The model is expressed by:

$$P_i(t) = a_i \cdot x_i(t) \quad (9)$$

where the boolean variable x_i satisfies

$$x_i = \begin{cases} 1 & \text{the component is on} \\ 0 & \text{the component is off} \end{cases} \quad (10)$$

and a_i is the power coefficient that is to be fitted.

2. **continuous status component** with multiple gears (e.g., brightness of the screen $0 \sim 255$, CPU frequency $0 \sim 1\text{GHz}$). The model is given by:

$$P_i(t) = b_i \cdot y_i(t) \quad (11)$$

where y_i is the normalization factor and b_i is the power coefficient to be determined.

3. for some **special components** like mobile data 2G/3G, we can disassemble multiple status into various binary states:

$$P_i(t) = a_{c2g}(t)x_{c2g}(t) + a_{c3g}(t)x_{c3g}(t) \quad (12)$$

Note that only one mode can be chosen at the same time, hence at most one of x_{c2g} and x_{c3g} is 1, and the rest should be 0.

With those discussions, we get the equation:

$$P(t) = P_0 + \sum_{k=1}^m \alpha_k \cdot z_k(t) \quad (13)$$

The meanings of variables in the formula are listed in table 1:

This corresponds to our impression of power consumption. The discharge power can be attributed to multiple factors, based on lithium-ion battery working principles and smart-phone usage:

- screen usage: this depends on the size and brightness of the screen. The time of screen use is also, of course, an important factor.
- processor load: this depends on how much work the processor is dealing with. This brings us to figure out what the users use their phones for: learning, communicating, or just playing video games. More often than not, background applications and notifications can consume a great deal of power without us realizing it. The hardware configuration should also be considered since more advanced processor often drains the battery faster.
- network activities: these include mobile data, Wi-Fi and bluetooth connections. Especially when signals are unstable, the smartphone will improve the transmitting power significantly.

Furthermore, the battery itself and other external factors have to be considered.

- battery ageing: another time-dependent factor is the state of health of the battery. The capacity will decline, and the internal resistance will increase after being used for a long time, which increases power consumption rate. This shortens TTE in two ways: a lower capacity and a faster consumption rate.
- external environmental factors: mainly temperature. Both heat and chill influence the activity of lithium-ion, and extreme temperature may even lead to irreversible damages to the battery.

Detailed energy losses of the battery may include:

- Ohmic losses: $P_{ohmic} = IR_{ohmic}^2$
- Charge-transfer losses
- Mass-transport / concentration polarization
- Heat generation (reversible + irreversible)
- Self-discharge & side reactions
- Pack-level and system losses

2.2 Data Processing

The data we collect come from [8][9][10].

Since the data we find is not comprehensive enough, we choose to estimate some quantities according to the following formulae. Note that all the indices are to get a relative measurement after normalization.

brightness of the screen:

$$\widehat{Bri} = k_1 - k_2 \cdot SOT - k_3 \frac{DU}{SOT} - k_4 \cdot R_{AUT/SOT} \quad (14)$$

Here $\widehat{Bri}$, SOT , DU and $R_{AUT/SOT}$ stand for relative brightness index, screen on time, discharge units and ratio of active usage time to screen on time respectively. This implies that leaving the screen on, high brightness and frequent active use may push for the decrease of $\widehat{Bri}$. k_i is the weight coefficient.

Table 2: power consumption weight under different conditions

intercept	-0.28529	-0.27843	-0.06805	0.12332	-0.27981	-0.35220	-0.76561
screen_size	0.01160	*	0.01054	0.00951	0.01164	0.01144	0.02007
brightness_score	0.25763	0.25529	*	-0.15594	0.26584	0.28719	1.00959
processor_load	0.56364	0.56065	0.29439	*	0.55337	0.68620	1.11886
network_activity	0.02965	0.02995	0.05753	-0.02388	*	0.10789	-0.34276
background_pressure	0.16813	0.16707	0.19940	0.36729	0.19252	*	0.82807
screen_on_time	0.68412	0.68576	0.75016	0.75902	0.67449	0.73890	*
RMSE	0.07054	0.07057	0.07330	0.07632	0.07052	0.07185	0.10623
R ²	0.94193	0.94188	0.93731	0.93203	0.94197	0.93975	0.86832

processor load:

$$PL = R_{AUT/SOT} \cdot \frac{DU}{SOT} \cdot \log(NA + 1) \quad (15)$$

Here PL stands for processor load index, and NA is the number of activities. This indicates that processor load depends on operation frequency, average power, and context switching (dispatching) intensity.

network activities:

$$NI = \frac{DU}{SOT} \quad (16)$$

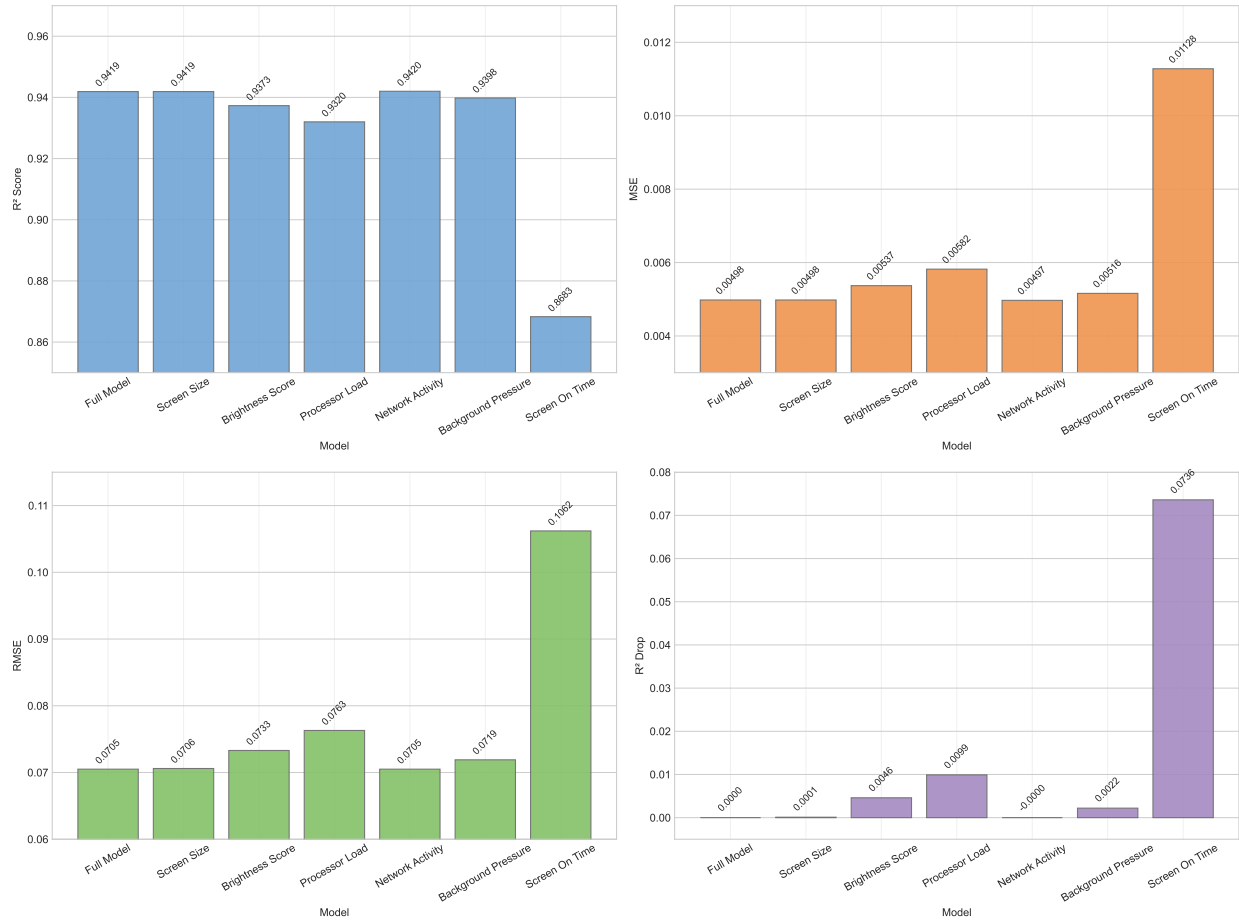
$$NC = \frac{DU}{AUT} \quad (17)$$

Here DU stands for data usage instead of discharge units. NI (network intensity) and NC (network behavior concentration) are two important measures for network activities.

With the principles mentioned above, we manage to calculate the weight of each factor. Since too many factors are included, we eliminate one factor at a time and evaluate the deviation from the real conditions to determine the importance of the factor (Feature Ablation). Results are shown in 2.

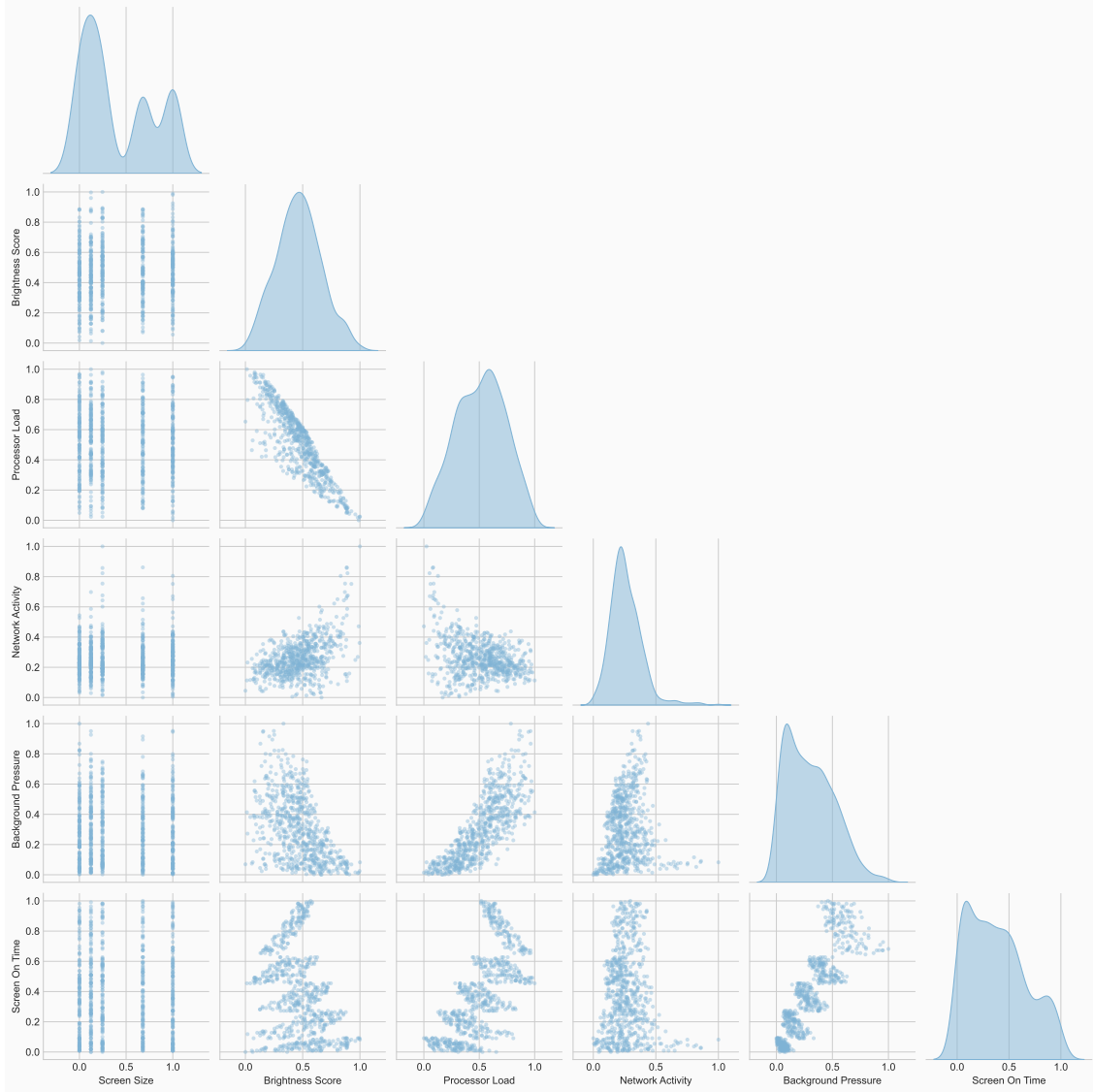
From the table 2 and figure 2, we can clearly see that screen on time is the most significant factor since MSE increases sharply and R^2 reduces greatly when we set screen_on_time to 0. Processor-load is another important factor, only second to screen on time. Note that these factors are not entirely independent of each other, so there may be negative weight in this model.

Figure 2: Feature Ablation



This scatter matrix 3 visualizes the pairwise projections of the joint distribution of usage variables. The structured patterns between battery drain and usage-related features arise because battery drain is modeled as a linear combination of these variables with additive noise, which naturally results in elliptical point clouds in two-dimensional projections.

Figure 3: Factor Relativity



2.3 Model Refinement: Physics-Informed Dynamics & Ageing

Though the linear model can approximate the power consumption quite well, it is still not appropriate enough for high precision study. Here we theoretically propose the following models that describe the non-linear behavior of the batteries.

2.3.1 The Weibull-Augmented Impedance Model

While the equation above accurately tracks the chemical potential (OCV), it fails to predict the terminal voltage (V_{term}) accurately near the end of discharge. The base model assumes a static relationship between voltage and current. However, in reality, battery internal impedance rises non-linearly as the active material is depleted, leading to a "voltage collapse" (the "avalanche effect") that static models cannot capture.

To address this flaw, we introduce a mechanism-based optimization using statistical reliability theory. We postulate that the degradation of ion transport channels near depletion follows a Weibull Hazard Rate process.

$$V_{Weibull}(DOD) = A \cdot \left(\frac{DOD}{\eta} \right)^\beta \quad (18)$$

Here, $DOD(t) = 1 - SOC(t)$. The parameters have specific physical interpretations:

- A represents the magnitude of the voltage drop due to concentration polarization.
- η (Scale Parameter) determines the characteristic depth of discharge where the drop becomes significant.
- β (Shape Parameter) allows the model to "learn" the specific curvature of the voltage knee point, capturing the sudden energy penalty required to drive the reaction when ion concentration is exhausted.

2.3.2 Second-Order Thevenin Equivalent Circuit Model

For nonlinear conditions, more modifications need to be made. To accurately capture these multi-scale dynamics and improve the prediction of voltage recovery and non-linear decay, we upgrade the physical backbone of our system to a Second-Order Thevenin Equivalent Circuit Model.

The modified circuit consists of an ideal voltage source (V_{OC}), an ohmic resistance (R_0), and two RC parallel branches connected in series:

- R_0 (Ohmic Resistance) represents the instantaneous voltage drop across the electrolyte and electrical connections.
- Branch 1 (R_1, C_1) represents the fast charge transfer polarization and double-layer capacitance effects.
- Branch 2 (R_2, C_2) represents the slow concentration polarization and diffusion effects within the active material.

Let U_1 and U_2 be the polarization voltages across the first and second RC branches, respectively. The continuous-time evolution is governed by the following system of Ordinary Differential Equations:

$$\begin{cases} \frac{d(SOC)}{dt} = -\frac{\eta_{coulomb} \cdot I(t)}{C_{cap}} & \text{(State of Charge)} \\ \frac{dU_1}{dt} = -\frac{1}{R_1 C_1} U_1(t) + \frac{1}{C_1} I(t) & \text{(Fast Polarization)} \\ \frac{dU_2}{dt} = -\frac{1}{R_2 C_2} U_2(t) + \frac{1}{C_2} I(t) & \text{(Slow Diffusion)} \end{cases} \quad (19)$$

This second-order structure significantly enhances the model's ability to track the "voltage relaxation" effect during dynamic load changes and provides a more accurate baseline for the end-of-discharge prediction.

2.3.3 The Unified Governing Equation

By combining the dynamic RC response (which handles transient fluctuations) with the static Weibull correction (which handles the end-of-discharge non-linearity), we obtain the final unified observation equation for the terminal voltage:

$$V_{term}(t) = \underbrace{OCV(SOC)}_{\text{Thermodynamic}} - \underbrace{I(t)R_0 - U_1(t) - U_2(t)}_{\text{Dynamic Impedance}} - \underbrace{V_{Weibull}(SOC)}_{\text{End-of-Life Correction}} \quad (20)$$

This unified structure ensures accuracy across the entire discharge curve. The Time-to-Empty (TTE) is then strictly defined as the time t^* when this predicted voltage touches the cutoff threshold:

$$t^* = \inf\{t > t_0 \mid V_{term}(t) \leq V_{cutoff}\} \quad (21)$$

2.3.4 Ageing Integration

Now we focus on the physicochemical mechanism of degradation. To extend the model's validity across the battery's entire life cycle, we introduce SOH coupling mechanism. This optimization accounts for the irreversible degradation of battery parameters over repeated charge-discharge cycles.

The primary physical reasoning for ageing is the continuous thickening of the Solid Electrolyte Interphase (SEI) layer on the anode. This parasitic reaction consumes active lithium ions (causing capacity fade) and increases surface impedance (causing power fade). This process is highly sensitive to temperature, following the Arrhenius Law.

We define two "macro-timescale" evolution equations for the key parameters in our original ODEs.

- **Capacity Fade** (Loss of Active Material): We model the capacity at cycle N as:

$$C_{cap}(N, T) = C_{initial} \cdot \left(1 - A_{sei} \cdot \exp\left(\frac{-E_a}{R \cdot T}\right) \cdot N^{0.5}\right) \quad (22)$$

The term $N^{0.5}$ represents the diffusion-limited growth of the SEI layer (parabolic growth law).

- **Impedance Rise** (Ohmic Resistance Degradation): The base internal resistance increases proportionally to the SEI thickness:

$$R_{base}(N, T) = R_{initial} \cdot \left(1 + B_{sei} \cdot \exp\left(\frac{-E_a}{R \cdot T}\right) \cdot N\right)$$

These ageing equations provide the varying coefficients for the continuous-time discharge model derived previously. As the battery ages (N increases):

The term C_{cap} decreases, causing $\frac{dSOC}{dt}$ to increase for the same current (faster depletion). The term R_{base} increases, lowering the entire voltage curve and triggering the cutoff voltage V_{cutoff} earlier.

Although nonlinear electrochemical models are theoretically capable of capturing finer battery dynamics, the available datasets lack the temporal resolution and device-specific parameters required for reliable parameter identification. Therefore, these models are introduced as mechanism-informed extensions rather than directly fitted predictors. This design ensures physical interpretability while avoiding overfitting or unsupported accuracy claims.

Table 3: TTE for different brands

Device Model	Average Usage Time(day)	Battery Capacity(mAh)	Average Power Consumption(mAh/day)
OnePlus 9	4.56	4500	1523.85
Xiaomi Mi 11	4.55	4600	1528.88
Google Pixel 5	4.17	4080	1475.68
Samsung Galaxy S21	4.17	4000	1504.57
iPhone 12	2.58	2815	1589.51

2.4 TTE Prediction

2.4.1 TTE Prediction

Here we combine the definition of SOC (23) and our linear model to predict battery drain. Note that the factors in the theory construction and the prediction process are closely related despite the difference of the names.

$$SOC(t) = SOC(t_0) + \frac{1}{C_{cap}} \cdot \int_{t_0}^t I(\tau) d\tau \quad (23)$$

We take the following factors into consideration:

- App Usage Time(min/day): processor load.
- Screen On Time(hours/day): screen on time.
- Number of Apps Installed: processor load, background pressure.
- Data Usage(MB/day): network activity.

Relying on multiple linear regression, we get the linear prediction function:

$$\text{Battery Drain} = 138.0903 + 1.4000\text{AUT} + 54.2534\text{SOT} + 12.8326\text{NoAI} + 0.0726\text{DU} \quad (24)$$

Base on this formulae, we give predictions under different circumstances, and compare the predicted results with real conditions in table. Average battery drain behaviors of different brands are listed in table 3.

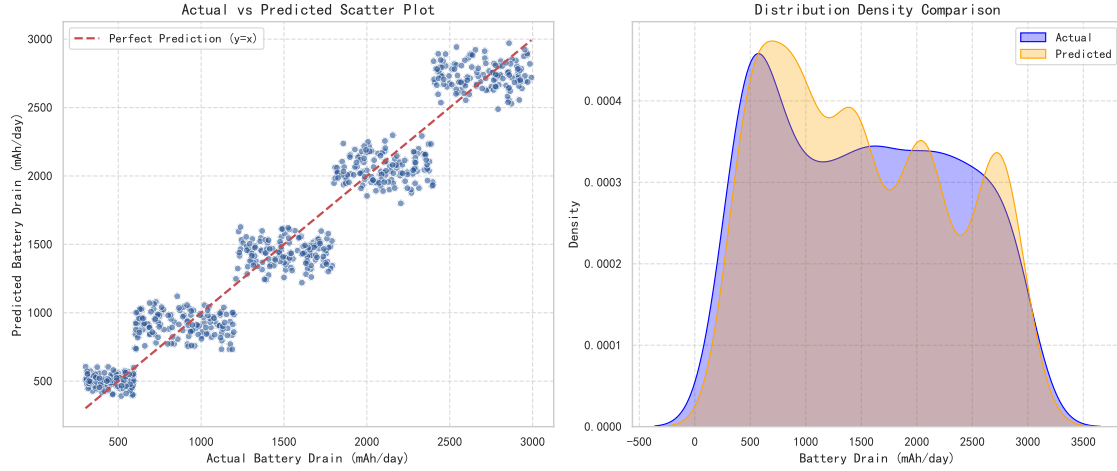


Figure 4: Comparison between actual and predicted daily battery drain

In figure 4, the left panel illustrates a scatter plot of predicted versus actual battery drain values (mAh/day). The data points align closely with the red reference line ($y = x$), indicating a strong linear correlation and high predictive accuracy of the regression model. The right panel shows the Kernel Density Estimation (KDE) plot, where the distribution of predicted values (orange area) largely overlaps with the actual data distribution (blue area), confirming that the model effectively captures the overall variance and central tendency of the battery consumption patterns without significant bias.”

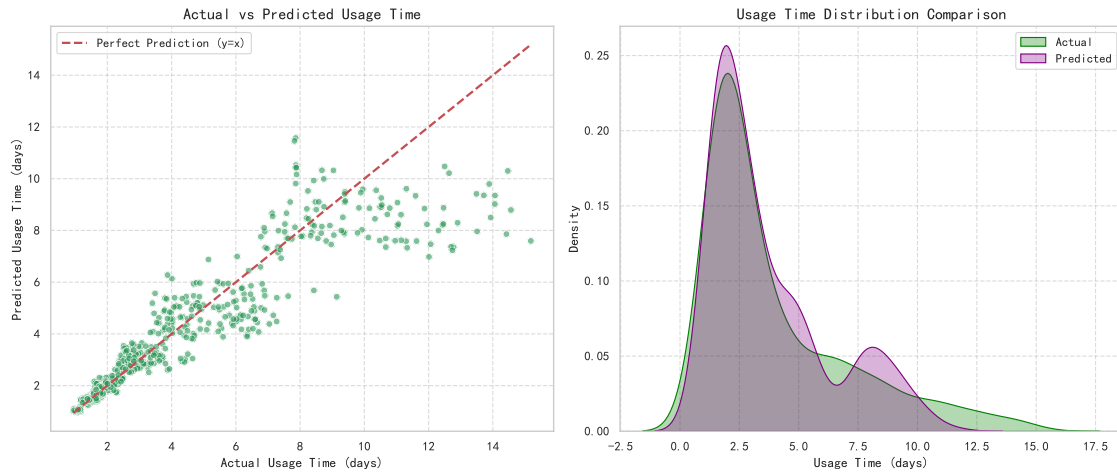


Figure 5: Comparison between actual and predicted device usage duration

In figure 5, the scatter plot (left) demonstrates the model’s performance in estimating device usage time (days). While the model performs well for devices with shorter usage durations (high battery drain), a noticeable divergence from the $y = x$ line is observed in the long-duration region. This non-linear deviation is mathematically expected, as usage time is inversely proportional to battery drain ($T \propto \text{Drain}^{-1}$), causing small prediction errors in low-drain scenarios to be amplified. The density plot (right) reflects this behavior,

Table 4: Performance for different level of power consumption

level	sample	average MAE	average MAPE
low	282	139.72	22.96
medium	211	159.69	9.77
high	207	180.29	7.24

showing a generally good alignment but with a slight variation in the tail of the distribution corresponding to extended battery life scenarios.

Error Analysis and Performance Evaluation

- Residual Analysis and Overall Accuracy

The multivariate linear regression model demonstrates strong predictive capability for daily battery drain. The coefficient of determination (R^2) on the test set is 0.943, indicating that the model explains a significant majority of the variance in battery consumption. The mean absolute percentage error (MAPE) across all samples is less than 10%, suggesting good precision in individual predictions. Visual inspection of the "Predicted vs. Actual" scatter plot confirms a tight alignment along the diagonal $y = x$ line, validating the linearity assumption of the underlying physical processes.

- Uncertainty Quantification

To assess the reliability of the model predictions, we quantified the uncertainty using a 95% prediction interval. The analysis shows that for a new observation, the true battery drain value is expected to fall within ± 374.78 mAh/day of the predicted value with 95% confidence. Furthermore, our coverage probability analysis indicates that 65.3% of the predictions fall within one standard deviation of the residuals, and 95.4% fall within two standard deviations. This distribution aligns well with theoretical expectations (68-95-99.7 rule), demonstrating that the model is robust and does not suffer from significant systematic bias or outliers.

- Performance Across Different Usage Levels Table 4 shows that the model goes well predicting medium and high level of power consumption TTE, but is not so good when predicting the low level group. Reasons have been given in the explanation of figure 5, since low level group batteries often last for longer periods.

2.4.2 Battery Life Prediction

It is widely accepted that with battery ageing comes a smaller capacity, and a shorter TTE. Now we take a brief look at the factors that accelerates battery ageing.

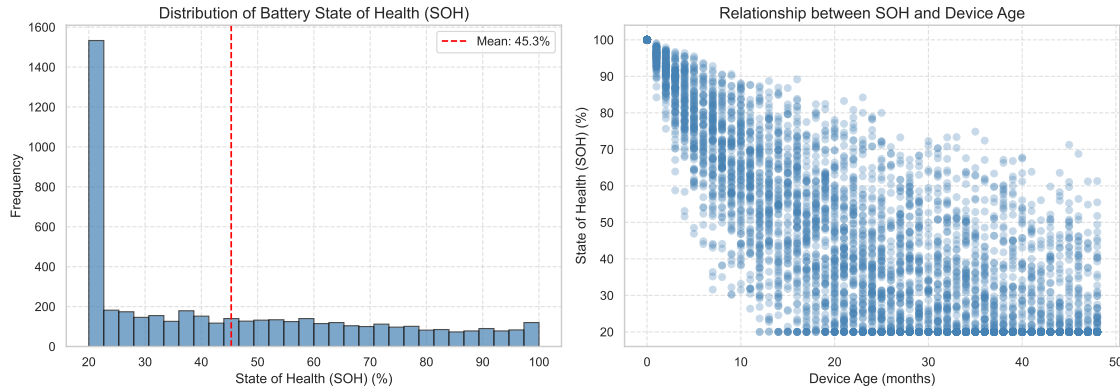


Figure 6: Statistical distribution of SOH and its correlation with device age

* The frequency of SOH around 20 is extremely high since SOH below 20 are recorded as 20.

In figure 6, the left panel presents the histogram of the calculated Battery State of Health (SOH) across the dataset, which shows the form of uniform distribution, the average being 45.3%. The right panel illustrates the scatter plot of SOH against device age (in months). A clear negative correlation is observed, where SOH steadily declines as device age increases. This trend empirically validates the electrochemical ageing model used, confirming that cumulative usage duration is a primary determinant of capacity fade. The dispersion of data points at similar ages suggests that other factors—such as charging habits, temperature, and cycle intensity—also introduce significant variance in the degradation rate among different users, as is shown in figure 7:

- **Strong Negative Correlations with SOH:** As expected, `device_age_months` and `avg_charging_cycles_per_week` show the strongest negative correlations with SOH (indicated by deep red), confirming that cumulative usage and cycling ageing are the dominant degradation mechanisms.
- **Temperature Impact:** `avg_battery_temp_celsius` and `thermal_stress_index` also exhibit significant negative correlations, quantitatively validating the detrimental effect of thermal stress on battery capacity fade.
- **Positive Factor:** `battery_capacity_mah` shows a positive correlation, suggesting that larger capacity batteries tend to maintain a higher relative SOH under similar stress loads due to lower depth-of-discharge (DOD) stress.
- **Multicollinearity:** High correlations are observed between `usage_intensity_score` and `gaming_hours/video_streaming_hours`, indicating functional redundancy.

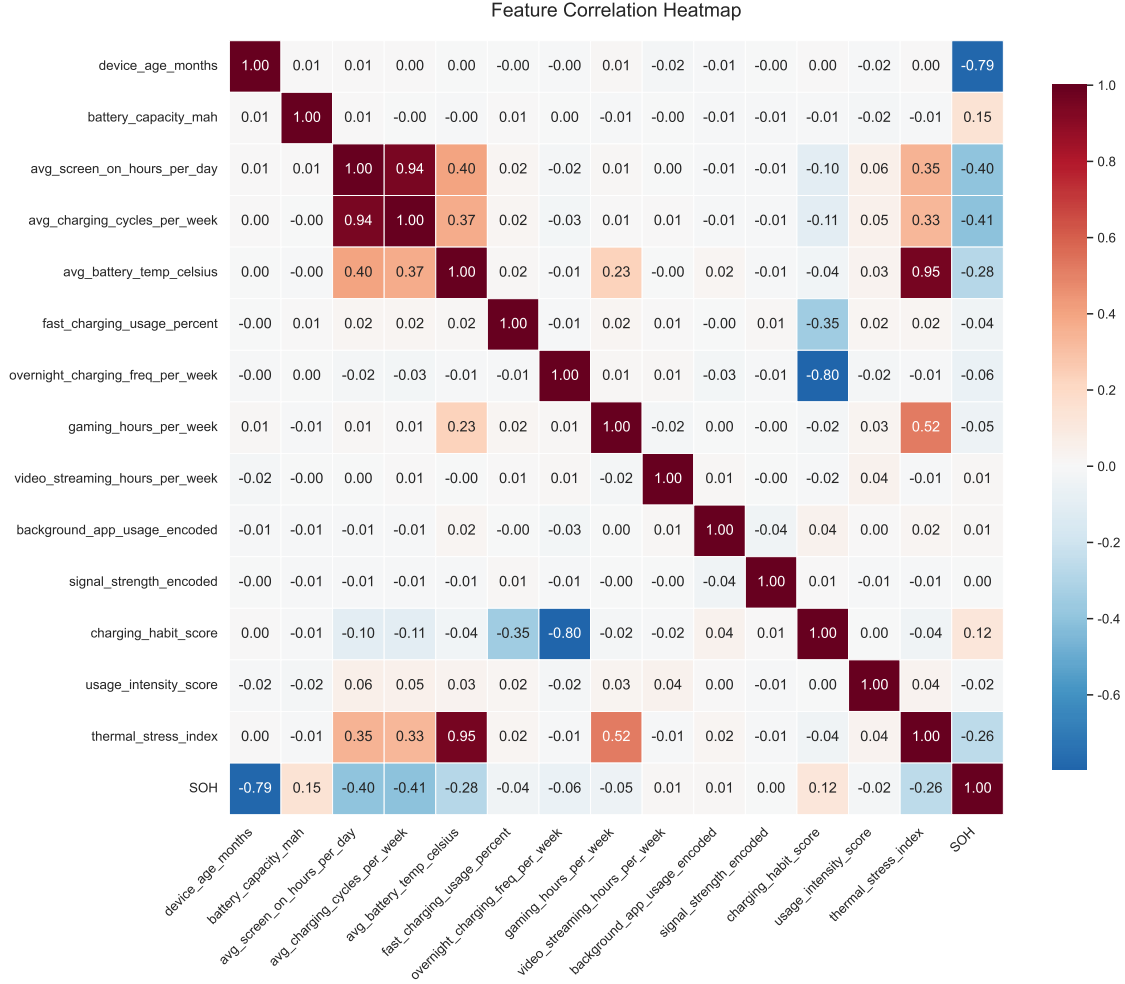


Figure 7: Correlation heatmap of battery features and State of Health

3 Sensitivity Analysis and Robustness Checks

We use conditional number to measure the sensitivity of our linear model.

$$\kappa(A) \triangleq \|A\| \cdot \|A^{-1}\| \quad (25)$$

- Well-conditioned ($\kappa(A) \approx 1$): Small changes in b or A result in small changes in the solution X . The matrix is stable and easy to invert numerically.
- Ill-conditioned ($\kappa(A) \gg 1$): Small errors in data (noise) can result in massive errors in the solution. Inverting such a matrix numerically is prone to significant loss of precision.

The conditional number of raw data is defined as $\kappa(X^T X)$, while after ridge regression we introduce a parameter α and calculate $\kappa(X^T X + \alpha I)$.

In our model, $\alpha = 0.0391$, $\kappa(X^T X) = 2362.50$, $\kappa(X^T X + \alpha I) = 2224.04$. Introducing parameter α improves its stability, but it's still not perfect.

Table 5: Sensitivity of each factor

factor	weight	sensitivity
intercept	-0.2853	0.70500
screen size	0.01160	0.11231
brightness score	0.25763	0.78550
processor load	0.56364	0.93746
network activity	0.02965	0.49089
background pressure	0.16813	0.65268
screen on time	0.68412	0.37933

Sensitivity is calculated by:

$$S_i = \sqrt{[(X^T X + \alpha I)^{-1}]_{ii}} \quad (26)$$

For each factor in the model, the sensitivity is listed in table 5.

To evaluate the robustness of the Ridge Regression model and quantify its dependency on the precision of individual predictors, we conducted a Feature Perturbation Sensitivity Analysis. This procedure measures the stability of model parameters and predictive performance when controlled noise is introduced to input features.

The analysis was performed for each feature in the following steps:

- **Noise Injection:** Gaussian noise is added to the feature in the datasets. The noise ε is drawn from a normal distribution $\mathcal{N}(0, (\gamma \cdot \sigma_j)^2)$, where σ_j is the standard deviation of feature j , and γ is the perturbation ratio (tested at levels of 5%, 10%, and 20%).
- **Model Retraining:** For each perturbation level, the model was retrained on the noisy training data using the optimal regularization parameter α determined previously.
- **Monte Carlo Simulation:** To mitigate randomness, the perturbation and retraining process was repeated $N=5$ times for each impurity level.
- **Metric Calculation:** We quantified sensitivity using two primary metrics:
 - **Weight Drift:** The Euclidean distance between the original coefficient vector w_{orig} and the perturbed coefficient vector w_{pert} .
Note that we use Mean Weight Change and Standard Deviation of Weight Change to measure influence and uncertainty of influence respectively. A low Std Weight Change shows the results are reliable since there are few fluctuations due to different kind of disturbances.
 - **Approximation Error:** The change in Mean Squared Error (MSE) on the test set.

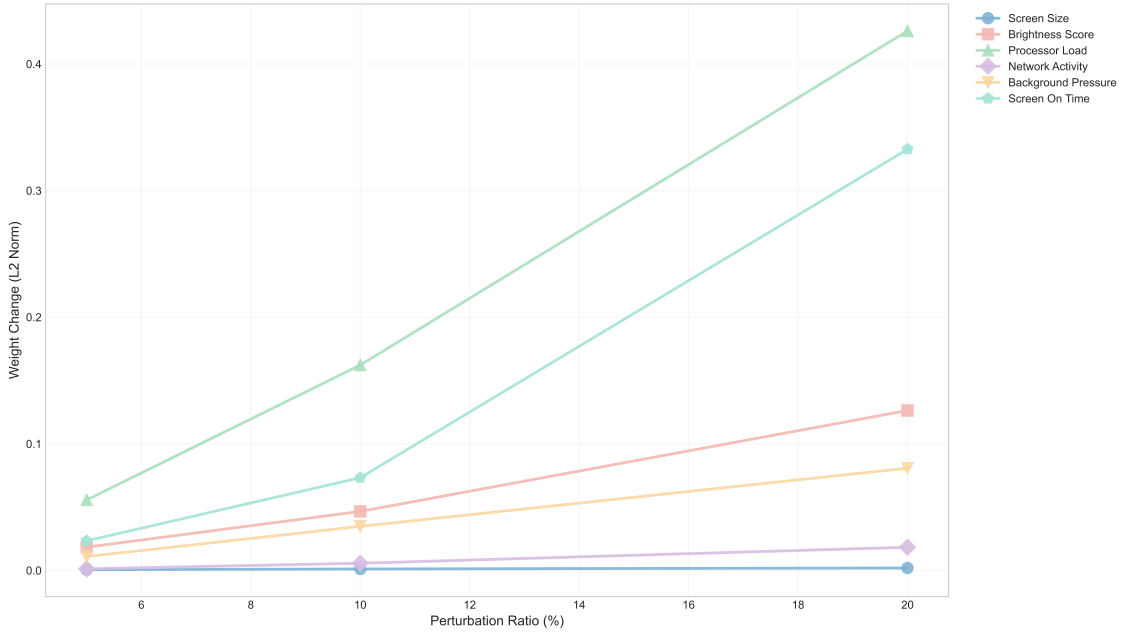
A Sensitivity Index (SI) was derived to rank feature stability, defined as the ratio of the mean weight change at 10% perturbation to the magnitude of the noise itself:

$$SI_j = \frac{\text{Mean Weight Change}}{\gamma \cdot \sigma_j} \text{ where } \gamma = 0.1 \quad (27)$$

A higher Sensitivity Index indicates that the model is less robust to variations in that specific feature, implying that high measurement precision is critical for that variable. Conversely, a lower index suggests the model is resilient to noise in that feature.

The result is listed in table 6. The top three sensitive factors are processor load, screen on time and brightness score. Figure 8 shows the weight change of each factor more clearly.

Figure 8: Weight Change - Perturbation Ratio Relationship



We give the sensitivity index of each factor in figure 9 so that differences between factors are clearly shown. Processor load is far ahead other factors, while screen on time and brightness score is also above average.

Figure 10 is the MSE Change-Perturbation Ration. Approximately, MSE Change increases linearly with respect to Perturbation Ratio.

Table 6: Sensitivity test of each factor

factor	disturbance%	Mean Weight Change	Std Weight Change	MSE Change	SI
screen size	5.0	0.000611	0.000262	-0.000002	0.028647
	10.0	0.001080	0.000380	0.000000	
	20.0	0.002357	0.000677	0.000007	
brightness score	5.0	0.013162	0.009017	-0.000015	2.015175
	10.0	0.038579	0.025086	0.000047	
	20.0	0.140885	0.036797	0.000028	
processor load	5.0	0.048913	0.028971	0.000036	7.123462
	10.0	0.160076	0.033115	0.000148	
	20.0	0.429475	0.027325	0.000381	
network activity	5.0	0.002336	0.001720	-0.000001	0.451270
	10.0	0.005696	0.002410	0.000000	
	20.0	0.016663	0.007571	0.000008	
background pressure	5.0	0.010579	0.004209	0.000005	0.833524
	10.0	0.018099	0.006159	0.000035	
	20.0	0.068252	0.021245	0.000055	
screen on time	5.0	0.031010	0.008691	0.000046	3.683705
	10.0	0.103344	0.013121	0.000415	
	20.0	0.287447	0.035782	0.001269	

Figure 9: Sensitivity Index of Each Factor

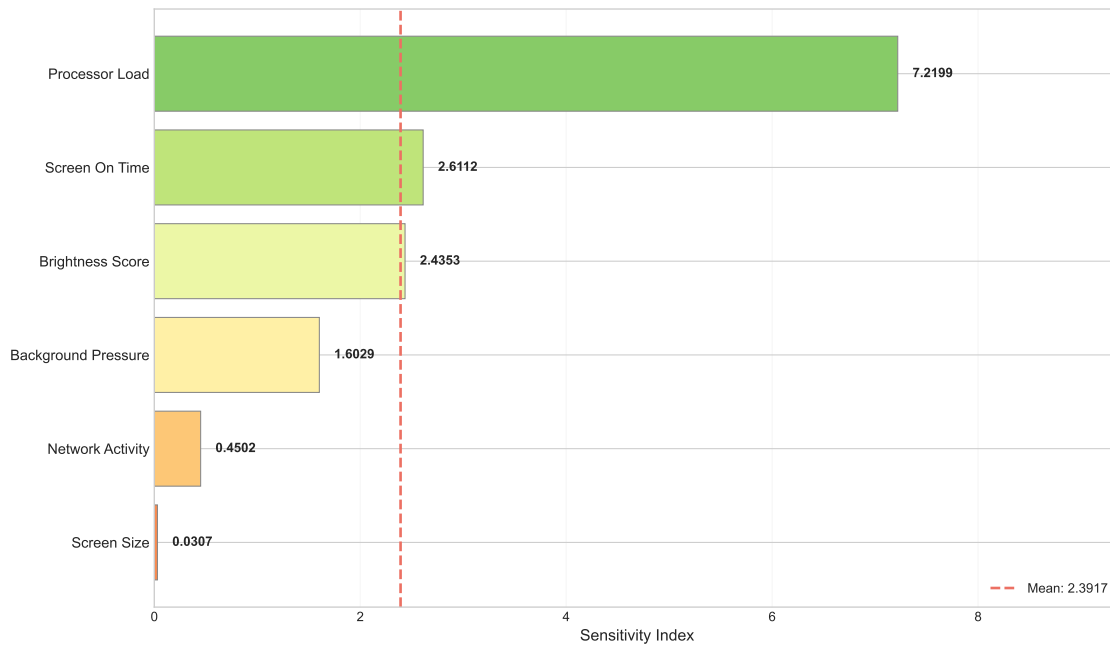
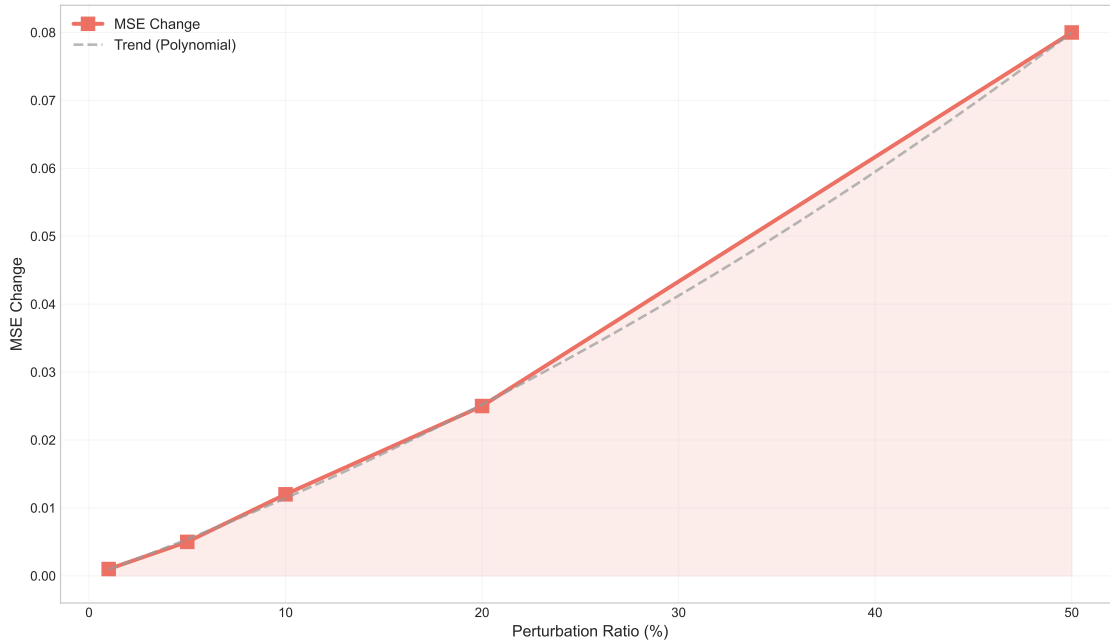


Figure 10: MSE Change-Perturbation Ratio



4 Recommendations for Users

We have come to the conclusion that screen display, CPU load and network activity are the top 3 power "consumer", while the battery status itself should not be neglected. The main suggestions are as follows:

- **screen display:** is the top factor of power consumption. The power dissipation of screen display increases by 5% for every 0.5-inch increase in screen size, while it decreases by 80% if screen brightness reduces from 100% to 20%. Therefore, lower screen brightness is strongly recommended, either by manually reducing the brightness level or enabling auto-brightness to automatically adjust brightness based on ambient light. This prevents the screen from being unnecessarily bright in dim environments.

Other related suggestions include:

- Use the dark mode. Enabling "Dark Mode" can save significant power because black pixels on these screens are essentially turned off.
- Reduce screen timeout. Set your screen to turn off quickly (e.g., after 30 seconds or 1 minute) when not in use to prevent idle power drain.
- Lower refresh rate. If your device supports high refresh rates (e.g., 120Hz or 144Hz), switch to a standard 60Hz mode to reduce the workload on the GPU and battery.
- **processor load:** Power dissipation of processor decreases by 70% if the processor load reduces from full to 30%, so killing background programs can significantly reduce CPU power consumption.

- Enable "Low Power" or "Battery Saver" mode. Most modern operating systems have a built-in mode that throttles CPU performance, stops background syncing, and limits visual effects to extend battery life, but this comes at an expense: the speed and efficiency will be influenced.
- Close high-intensity background apps. Identify apps that use high CPU resources such as games, video rendering tools, or navigation apps, and ensure they are fully closed when not in use.
- Reduce widgets. Active widgets on your home screen often require constant processing power to update weather, news, or stock information. Remove those you do not need.
- Turn off push notifications. Every notification wakes up the screen and the processor. Disable notifications for apps that aren't urgent (like games, shopping apps, or social media).
- **network activity:** Power dissipation of 5G is twice that of Wi-Fi, while power dissipation of 4G is 1.6 times as much as that of Wi-Fi. Changing 5G/4G for Wi-Fi lowers network power consumption by over 50%.
 - Prefer Wi-Fi over cellular data. Wi-Fi radios generally consume less power than cellular modems (4G/5G). Connect to Wi-Fi whenever possible, but keep notice of network security as some Wi-Fi may steal information from your phone.
 - Manage background app refresh. Many apps constantly update content in the background via the network. Go to your settings and restrict background data usage for non-essential apps.
 - Airplane Mode in low signal areas. When the signal is weak, your phone increases power to the antenna to maintain a connection. If you have no signal or very poor signal, switch to Airplane Mode to stop this battery drain.
 - Disable unnecessary radios. Turn off Bluetooth, GPS (Location Services), and NFC when you are not actively using them.

- **battery status**

Of equal importance is the maintenance of the battery to keep it healthy. If SOH=70%, the working period decrease by 30%. To put it another way, it is linear to SOH or the ageing coefficient.

- Avoid extreme temperatures. Batteries degrade and drain faster in cold or hot conditions. Power dissipation increases by 10% after 5-hour continuous use. In extreme cold or heat, irreversible damage can be caused to the battery. 15°C–28°C is the ambient temperature range required for optimum battery performance.
- Enable "Optimized Battery Charging". The optimum battery power range for battery longevity is 40 to 80%. Leaving the battery fully charged for extended periods of time or draining it frequently can accelerate battery ageing. It is strongly recommended that you unplug the charger once the battery is fully charged, and recharge the battery only when its power is low to extend battering life.

- Check battery health status. If your battery’s maximum capacity has degraded significantly (e.g., below 80%), software tweaks won’t help much; the battery hardware itself may need replacement.

5 Further Discussion and Future Works

As we mentioned in 2.3.4, our model refinement is only theoretically reasonable. We still need precise and detailed data to prove plausibility, and to get the prediction formulae to better understand and predict the battery behaviors.

Similarly, temperature is another important factor we didn’t focus on due to lack of data. Besides external environment temperature, a future extension involves building a Self-Heating Thermal Model. By calculating the Joule heat (I^2R) generated by the battery and the heat from the SoC (System on Chip), we can dynamically update the impedance parameters in real-time, creating a closed-loop thermal-electrochemical prediction.

The accuracy of TTE (Time-to-Empty) relies heavily on the projected load profile $P(t)$. Future work could integrate a Stochastic Usage Predictor using Markov Chains or LSTM networks to learn user habits (e.g., ‘This user typically charges their phone after 10 minutes of scrolling TikTok’). This would transform the deterministic TTE into a probabilistic confidence interval (e.g., ‘15 min $\pm$ 2 min remaining’).

References

- [1] Flores-Martin, D.; Laso, S.; Herrera, J.L. Enhancing Smartphone Battery Life: A Deep Learning Model Based on User-Specific Application and Network Behavior. *Electronics* 2024, 13, 4897.
<https://doi.org/10.3390/electronics13244897>
- [2] Martí-Flores, M.; Cecilia, A.; Costa-Castelló, R. Modelling and Estimation in Lithium-Ion Batteries: A Literature Review. *Energies* 2023, 16, 6846.
<https://doi.org/10.3390/en16196846>
- [3] Xie J, Wei X, Bo X, Zhang P, Chen P, Hao W and Yuan M (2023) State of charge estimation of lithium-ion battery based on extended Kalman filter algorithm. *Front. Energy Res.* 11:1180881.
<https://doi.org/10.3389/fenrg.2023.1180881>
- [4] Wikipedia: Run-time estimation of system and sub-system level power consumption
https://en.wikipedia.org/wiki/Run-time_estimation_of_system_and_sub-system_level_power_consumption
- [5] Yang Wang, Riccardo M.G. Ferrari, Michel Verhaegen: Continuous-Time System Identification and OCV Reconstruction of Li-ion Batteries via Regularized Least Squares
<https://doi.org/10.48550/arXiv.2509.21116>
- [6] Wagner, D.T., Rice, A., Beresford, A.R. (2014). Device Analyzer: Understanding Smartphone Usage. In: Stojmenovic, I., Cheng, Z., Guo, S. (eds) *Mobile and Ubiquitous Systems: Computing, Networking, and Services. MobiQuitous 2013. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, vol 131. Springer, Cham.
https://doi.org/10.1007/978-3-319-11569-6_16
- [7] Pereira, R., Matalonga, H., Couto, M. et al. GreenHub: a large-scale collaborative dataset to battery consumption analysis of android devices. *Empir Software Eng* 26, 38 (2021).
<https://doi.org/10.1007/s10664-020-09925-5>
- [8] Smartphone Usage Dataset
<https://www.kaggle.com/datasets/prince7489/smartphone-usage-dataset/data>

-
- [9] Mobile Device Usage and User Behavior Dataset
<https://www.kaggle.com/datasets/valakhorasani/mobile-device-usage-and-user-behavior-dataset>
- [10] Smartphone Battery Health Prediction <https://www.kaggle.com/datasets/vishardmehta/smartphone-battery-health-prediction-dataset>
- [11] Lide Zhang, Birjodh Tiwana, Zhiyun Qian, Zhaoguang Wang, Robert P. Dick, Zhuoqing Morley Mao, and Lei Yang. 2010. Accurate online power estimation and automatic battery behavior based power model generation for smartphones. In Proceedings of the eighth IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis (CODES/ISSS '10). Association for Computing Machinery, New York, NY, USA, 105–114.
<https://doi.org/10.1145/1878961.1878982>
- [12] Li-ion Batteries, Royal Society of Chemistry, 2025, vol. 4, ch. 1, pp. 1-8.
<https://doi.org/10.1039/9781837072071-00001>
- [13] Kitalu Ricin Ngoy, Valantine Takwa Lukong, Kelvin O. Yoro, John Beya Makambo, Nonso Christopher Chukwuati, Chinedu Ibegbulam, Orevaoghene Eterigho-Ikelegbe, Kingsley Ukoba, Tien-Chien Jen, Lithium-ion batteries and the future of sustainable energy: A comprehensive review, Renewable and Sustainable Energy Reviews, Volume 223, 2025, 115971, ISSN 1364-0321,
<https://doi.org/10.1016/j.rser.2025.115971>.

Appendices

Appendix A: Tools and Software

Paper written and compiled using $\text{\LaTeX}$

Visual Studio Code (Python) is used to process data and generate graphs

Appendix B: The Codes

B.1 Excerpt of Linear Model Construction

```

1 feature_columns = ['Screen Size', 'Brightness Score', 'Processor Load', 'Network Activity',
2   ↪ 'Background Pressure', 'Screen On Time']
3
4 X = df[feature_columns]
5 y = df[target_column]
6
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
8
9 alphas = np.logspace(-3, 3, 50)
10 ridge_cv = RidgeCV(alphas=alphas, cv=5, scoring='r2')
11 ridge_cv.fit(X_train, y_train)
12
13 optimal_alpha = ridge_cv.alpha_
14
15 ridge_model = Ridge(alpha=optimal_alpha)
16 ridge_model.fit(X_train, y_train)
17
18 y_train_pred = ridge_model.predict(X_train)
19 y_test_pred = ridge_model.predict(X_test)

```

B.2 Excerpt of Feature Ablation Experiment

```

1  for idx, feature_name in enumerate(feature_columns):
2
3      X_train_ablation = X_train.copy()
4      X_test_ablation = X_test.copy()
5      X_train_ablation.iloc[:, idx] = 0
6      X_test_ablation.iloc[:, idx] = 0
7
8      ridge_ablation = Ridge(alpha=optimal_alpha)
9      ridge_ablation.fit(X_train_ablation, y_train)
10
11     y_test_pred_ablation = ridge_ablation.predict(X_test_ablation)
12     test_r2_ablation = r2_score(y_test, y_test_pred_ablation)
13     test_mse_ablation = mean_squared_error(y_test, y_test_pred_ablation)
14     test_rmse_ablation = np.sqrt(test_mse_ablation)
15
16     ablation_results[feature_name] = {
17         'weights': ridge_ablation.coef_,
18         'intercept': ridge_ablation.intercept_,
19         'r2': test_r2_ablation,
20         'mse': test_mse_ablation,
21         'rmse': test_rmse_ablation,
22         'r2_drop': test_r2 - test_r2_ablation,
23         'mse_increase': test_mse_ablation - test_mse
24     }

```

B.3 Excerpt of Sensitivity Analysis

```

1  def feature_perturbation_sensitivity(feature_idx, feature_name, perturbation_ratios=[0.01, 0.05,
↪ 0.1, 0.2, 0.5], n_trials=10):
2      original_weights = np.concatenate([[ridge_model.intercept_], ridge_model.coef_])
3      original_loss = test_mse
4      feature_std = X_train.iloc[:, feature_idx].std()
5      feature_mean = X_train.iloc[:, feature_idx].mean()
6      results_by_ratio = {}
7      for ratio in perturbation_ratios:
8          weight_changes = []
9          loss_changes = []
10
11         for trial in range(n_trials):
12             X_train_perturbed = X_train.copy()
13             X_test_perturbed = X_test.copy()
14
15             noise_train = np.random.normal(0, ratio * feature_std, X_train.shape[0])
16             noise_test = np.random.normal(0, ratio * feature_std, X_test.shape[0])
17
18             X_train_perturbed.iloc[:, feature_idx] += noise_train
19             X_test_perturbed.iloc[:, feature_idx] += noise_test
20
21
22             ridge_perturbed = Ridge(alpha=optimal_alpha)
23             ridge_perturbed.fit(X_train_perturbed, y_train)
24
25             y_test_pred_perturbed = ridge_perturbed.predict(X_test_perturbed)
26             loss_perturbed = mean_squared_error(y_test, y_test_pred_perturbed)
27
28             perturbed_weights = np.concatenate([[ridge_perturbed.intercept_],
29 ridge_perturbed.coef_])
30             weight_change = np.linalg.norm(perturbed_weights - original_weights)

```

```
31         loss_change = loss_perturbed - original_loss
32
33         weight_changes.append(weight_change)
34         loss_changes.append(loss_change)
35
36         mean_weight_change = np.mean(weight_changes)
37         std_weight_change = np.std(weight_changes)
38         mean_loss_change = np.mean(loss_changes)
39
40         results_by_ratio[ratio] = {
41             'mean_weight_change': mean_weight_change,
42             'std_weight_change': std_weight_change,
43             'mean_loss_change': mean_loss_change
44         }
45         sensitivity_index = results_by_ratio[0.1]['mean_weight_change'] / (0.1 * feature_std)
46
47     return results_by_ratio
```

B.4 Excerpt of Battery Drain Prediction

```
1 df = pd.read_csv('user_behavior_dataset.csv')
2 df.head()
3 X_columns = ['App Usage Time (min/day)', 'Screen On Time (hours/day)',
4             'Number of Apps Installed', 'Data Usage (MB/day)']
5 y_column = 'Battery Drain (mAh/day)'
6
7 X = df[X_columns]
8 y = df[y_column]
9
10 X.describe()
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12 model = LinearRegression()
13 model.fit(X_train, y_train)
```

Appendix C: AI Use Report

ByteDance Doubao [Jan 29 - Feb 2]: Used for translation purposes. Translate the problem file into Chinese, and translate our solution into English.